EECS 3311 Software Design

Github: https://github.com/nurjahan-shiah/Software-Design

Winter 2026

Sections Z

Group 11

Nurjahan Ahmed Shiah
Joseph Ajeigbe
Jessica Buentipo
David Ekeng
Iyinoluwa Olu-Alabi

Task 1a: Justification for the Six Design Patterns

Our system redesign for Deliverable 2 required introducing design patterns that improve the maintainability, extensibility, and correctness of the lab equipment reservation platform. The following six patterns were selected based on a direct analysis of the system requirements (Req1 - Req10) and the structure of the class diagram produced in Deliverable 1. Each pattern addresses a specific design problem that would otherwise require significant code duplication or tight coupling between system components.

1. Factory Pattern

Pattern Type: Creational
Key Classes: UserFactory, User, Student, Faculty, Researcher, Guest, LabManager
Requirement: Req1, Req2

Problem
The system supports six distinct user types: Student, Faculty, Researcher, Guest, LabManager, and HeadLabCoordinator, each with different hourly fee rates and permissions. Without a factory, every component that creates a user would need to know the exact subclass to instantiate, leading to duplicated conditional logic spread across the codebase.

Solution
UserFactory centralizes user object creation. It accepts a userType string and returns the correct User subclass. The calling code (LoginPanel, UserDAO) only depends on the abstract User type, not on any concrete subclass. This is the classic Factory Method pattern, where object creation is delegated to a factory class.

How It Is Used in Our System

- **LoginPanel** calls UserFactory.getUserType(type) during registration without knowing which subclass to create.
- **UserDAO** uses the same factory during CSV loading to reconstruct the correct subclass from the stored type string.
- Each subclass (Student, Faculty, etc.) overrides getFeeRate() to return its hourly fee ($10, $15, $20, $30), which the factory produces automatically.
- Adding a new user type in the future only requires adding a new subclass and one branch in UserFactory : no other files change.

Code Evidence

```
UserFactory factory = new UserFactory();
User newUser = factory.getUserType("STUDENT");  // returns new Student()
```

```
// Student.getFeeRate() returns 10.0
// Faculty.getFeeRate() returns 15.0
```

2. Singleton (BookingSystem) Pattern

Pattern Type: Creational
Key Classes: BookingSystem, Reservation
Requirement: Req3, Req4, Req8, Req9

Problem
The system needs a single, shared registry of all active reservations that is accessible from both the GUI panels and the command layer. If multiple BookingSystem instances existed, checking for booking conflicts or cancelling future bookings would be unreliable because each instance would have an incomplete view of the active reservations.

Solution
BookingSystem uses the Singleton pattern with a private constructor and a static getInstance() method. The single instance is created eagerly at class load time. All booking operations: adding a booking, checking active bookings, and cancelling future bookings, go through this one shared instance.

How It Is Used in Our System

- **UserDashboardPanel** calls BookingSystem.getInstance().addBooking(reservation) every time a booking is confirmed.
- **ManagerDashboardPanel** calls BookingSystem.getInstance().checkActiveBookings(equipmentID) before changing equipment status.
- **cancelFutureBookings(id)** is called when equipment is marked for maintenance to cancel all upcoming reservations.
- Because there is only one instance, all panels always see the same, complete booking list.

Code Evidence

```
private static BookingSystem bsInstance = new BookingSystem();
public static BookingSystem getInstance() {
return bsInstance;
}
private BookingSystem() { ...
        } // private constructor prevents new BookingSystem()
// Usage: BookingSystem.getInstance().addBooking(r);
```

3. Singleton (HeadLabCoordinatorSingleton) Pattern

Pattern Type: Creational
Key Classes: HeadLabCoordinatorSingleton, LabManager
Requirement: Req2

Problem
Req2 explicitly states that only one Head Lab Coordinator exists in the system, and only that person can auto-generate Lab Manager accounts. Without a Singleton, it would be possible to instantiate multiple coordinator objects, which would violate this constraint and allow unauthorized account creation.

Solution
HeadLabCoordinatorSingleton uses lazy initialization: the instance is only created when getInstance() is first called. The private constructor prevents any other code from creating a second coordinator. The generateManagerAccount() method is the only way to create a LabManager, ensuring the business rule in Req2 is enforced at the design level, not just checked in the GUI.

How It Is Used in Our System

- **CoordinatorDashboardPanel** calls HeadLabCoordinatorSingleton.getInstance().generateManagerAccount(...) to create a new manager.
- If the logged-in user is not a COORDINATOR, the GUI blocks the action before even reaching the Singleton.
- The **generateManagerAccount()** method creates a LabManager with a system-generated temporary password.
- Lazy initialization means the singleton is only created when the coordinator actually logs in, not at startup.

Code Evidence

```
private static HeadLabCoordinatorSingleton instance;
public static HeadLabCoordinatorSingleton getInstance() {
        if (instance == null) {
        instance = new HeadLabCoordinatorSingleton();
        }
        return instance;
}
public LabManager generateManagerAccount(String id, String name, ...) {
... }
```

4. Observer Pattern

Pattern Type: Behavioural
Key Classes: EquipmentObserver, EquipmentSubject, EquipmentContext, UserNotificationObserver
Requirement: Req5, Req6

Problem

When a manager changes the equipment status to DISABLED or MAINTENANCE, all users who have active bookings on that equipment must be notified. Without the Observer pattern, EquipmentContext would need a direct reference to UserDAO and the notification logic, creating tight coupling between equipment management and user notification.

Solution

EquipmentContext acts as the Subject. It maintains a list of EquipmentObserver instances. When setState() is called (triggered by enable(), disable(), or markUnderMaintenance()), it calls notifyObservers(), which dispatches the update to all registered observers. UserNotificationObserver is the concrete Observer: it receives the equipmentID and newStatus and generates a notification message for each affected user.

How It Is Used in Our System

- Before changing equipment status, the ManagerDashboardPanel builds a list of users with active bookings on that equipment.
- A **UserNotificationObserver** is created with that user list and registered on the EquipmentContext.
- When the status changes, EquipmentContext.setState() automatically calls notifyObservers().
- UserNotificationObserver.update() runs for each affected user and logs the notification message.
- The manager dashboard displays the notification log so the manager can confirm who was notified.
- EquipmentContext also implements the EquipmentSubject interface, keeping Subject and State responsibilities cleanly separated.

Code Evidence

```
UserNotificationObserver notifier = new UserNotificationObserver(affectedUsers);
context.registerObserver(notifier);
context.disable();  // triggers setState() → notifyObservers() → notifier.update()
// In UserNotificationObserver:
public void update(String equipmentID, String newStatus) {
        if (newStatus.equals("DISABLED") || newStatus.equals("MAINTENANCE")) {
        // generate notification for each affected user
        }
}
```

5. Strategy Pattern

Pattern Type: Behavioural
Key Classes: PaymentStrategy, CreditCardPayment, DebitPayment, InstitutionalPayment, GrantPayment, PaymentProcessor
Requirement: Req10

Problem
Req10 requires support for four payment methods: credit card, debit, institutional account, and research grant. Each method has a different processing logic. Without Strategy, a single processPayment() method would contain a long if-else chain checking the payment type, making it difficult to add new payment methods or test each independently.

Solution
PaymentStrategy is an interface with a single pay(PaymentTransaction) method. Each payment method is a separate class implementing this interface. PaymentProcessor holds a reference to any PaymentStrategy and delegates payment processing to it. The calling code (UserDashboardPanel) selects the correct strategy based on the user's choice and passes it to the processor: the processor itself never knows which payment method is being used.

How It Is Used in Our System

- **UserDashboardPanel** reads the selected payment method from the payCombo dropdown.
- A switch statement selects the correct PaymentStrategy implementation.
- PaymentProcessor is created with that strategy, and processPayment(txn) is called.
- Each strategy (CreditCardPayment, DebitPayment, etc.) sets the transaction status to SUCCESS and returns true.
- A new payment method can be added by creating a new class implementing PaymentStrategy: no existing code changes.

Code Evidence

```
PaymentStrategy strategy = new CreditCardPayment();  // or DebitPayment, etc.
PaymentProcessor processor = new PaymentProcessor(strategy);
boolean paid = processor.processPayment(txn);
// PaymentStrategy interface:
public interface PaymentStrategy {
        boolean pay(PaymentTransaction transaction);
}
```

6. State Pattern

Pattern Type: Behavioural
Key Classes: EquipmentState, EquipmentContext, AvailableState, DisabledState, MaintenanceState
Requirement: Req6, Req7

Problem

Equipment can be in one of three states: AVAILABLE, DISABLED, or MAINTENANCE. Each state has different behaviour: for example, you cannot disable equipment that is already disabled, and enabling equipment from MAINTENANCE is valid, but enabling it from AVAILABLE is not. Without the State pattern, EquipmentContext would need nested if-else or switch statements to check the current state before every transition, which becomes difficult to maintain as new states are added.

Solution

EquipmentState is an interface with three methods: enable(), disable(), and markUnderMaintenance(). Each state class implements this interface and defines the correct behaviour for transitions from that state. EquipmentContext holds a reference to the current state and delegates all state-change calls to it. When a transition occurs, the current state replaces itself with the new state by calling context.setState(newState).

How It Is Used in Our System

- AvailableState.disable() transitions to DisabledState; AvailableState.enable() prints 'already enabled'.
- DisabledState.enable() transitions back to AvailableState.
- MaintenanceState.enable() transitions to AvailableState (maintenance completed).
- When EquipmentContext.setState() is called, it also triggers notifyObservers() (Observer pattern integration).
- ManagerDashboardPanel only calls context.enable(), context.disable(), or context.markUnderMaintenance(): It never checks the current state directly.

Code Evidence

```
// AvailableState:
public void disable(EquipmentContext context) {
        context.setState(new DisabledState());
}
// DisabledState:
public void enable(EquipmentContext context) {
        context.setState(new AvailableState());
}
// EquipmentContext delegates to current state:
public void disable() { currentState.disable(this); }
```

7. Command Pattern

Pattern Type: Behavioural
Key Classes: Command, ReservationService, ReserveCommand, CancelCommand, ExtendCommand, ForfeitDepositCommand, ModifyCommand
Requirement: Req4, Req8, Req9

Problem
Booking operations: reserve, cancel, extend, and forfeit, need to be encapsulated as objects so that the GUI does not directly manipulate the model. Without Command, UserDashboardPanel would call reservation.setStatus('CANCELLED') and other model methods directly, creating a direct dependency between the GUI and model layers and making it difficult to add undo functionality or log operations.

Solution
Command is an interface with a single execute() method. Each booking action is a concrete command class: ReserveCommand, CancelCommand, ExtendCommand, ForfeitDepositCommand. Each command holds a reference to the ReservationService (the receiver) and the Reservation it operates on. The GUI creates the command and calls execute(): it does not call the service directly.

How It Is Used in Our System

- ReserveCommand calls reservationService.reserve(reservation) which sets the status to CONFIRMED.
- CancelCommand calls reservationService.cancel(reservation) which calls reservation.cancel().
- ExtendCommand calls reservationService.extend(reservation, newEndTime).
- ForfeitDepositCommand calls reservationService.forfeitDeposit(reservation) which marks the deposit as forfeited.
- UserDashboardPanel creates and executes commands: new ReserveCommand(svc, r).execute()
- ReservationService acts as the receiver: it contains the actual business logic for each operation.

Code Evidence

```
Command cmd = new ReserveCommand(reservationService, reservation);
cmd.execute();  // calls reservationService.reserve(reservation)

Command cancel = new CancelCommand(reservationService, reservation);
cancel.execute();  // calls reservationService.cancel(reservation)
```

```
// Command interface:
public interface Command {
void execute();
}
```

Summary: Six Design Patterns

The table below summarises all six patterns and their mapping to the system requirements.

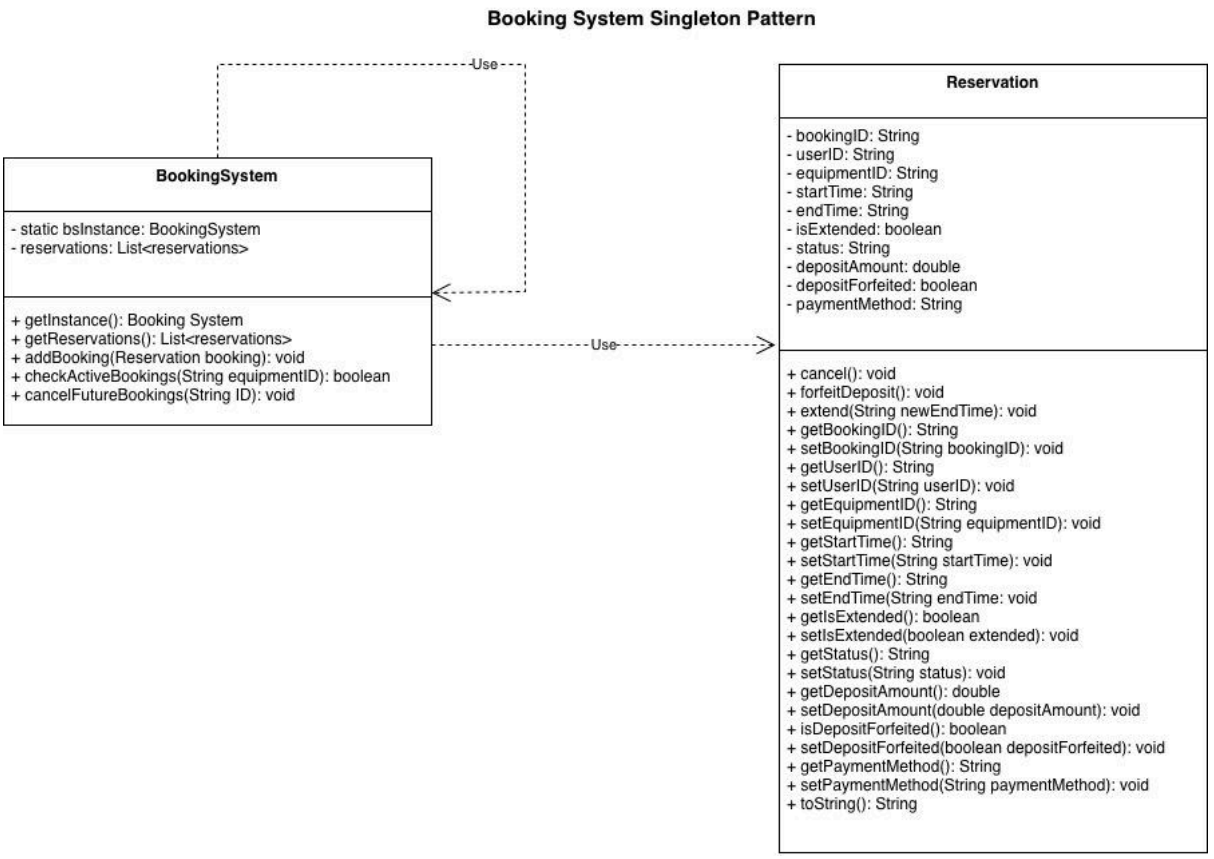
Pattern	Type	Key Classes	Requirement(s)
Factory	Creational	UserFactory	Req1, Req2
Singleton	Creational	BookingSystem	Req3, Req4, Req8, Req9
Singleton	Creational	HeadLabCoordinatorSingleton	Req2
Observer	Behavioural	EquipmentContext, UserNotificationObserver	Req5, Req6
Strategy	Behavioural	PaymentStrategy + 4 implementations	Req10
State	Behavioural	EquipmentContext, 3 State classes	Req6, Req7
Command	Behavioural	Command + 5 command classes	Req4, Req8, Req9

Note: The assignment requires a minimum of six design patterns. Our implementation includes seven pattern applications across six distinct pattern types (Singleton is applied twice in different contexts). This strengthens the justification because each application solves a different problem: BookingSystem ensures a single booking registry, while HeadLabCoordinatorSingleton enforces the Req2 business rule that only one coordinator can exist.

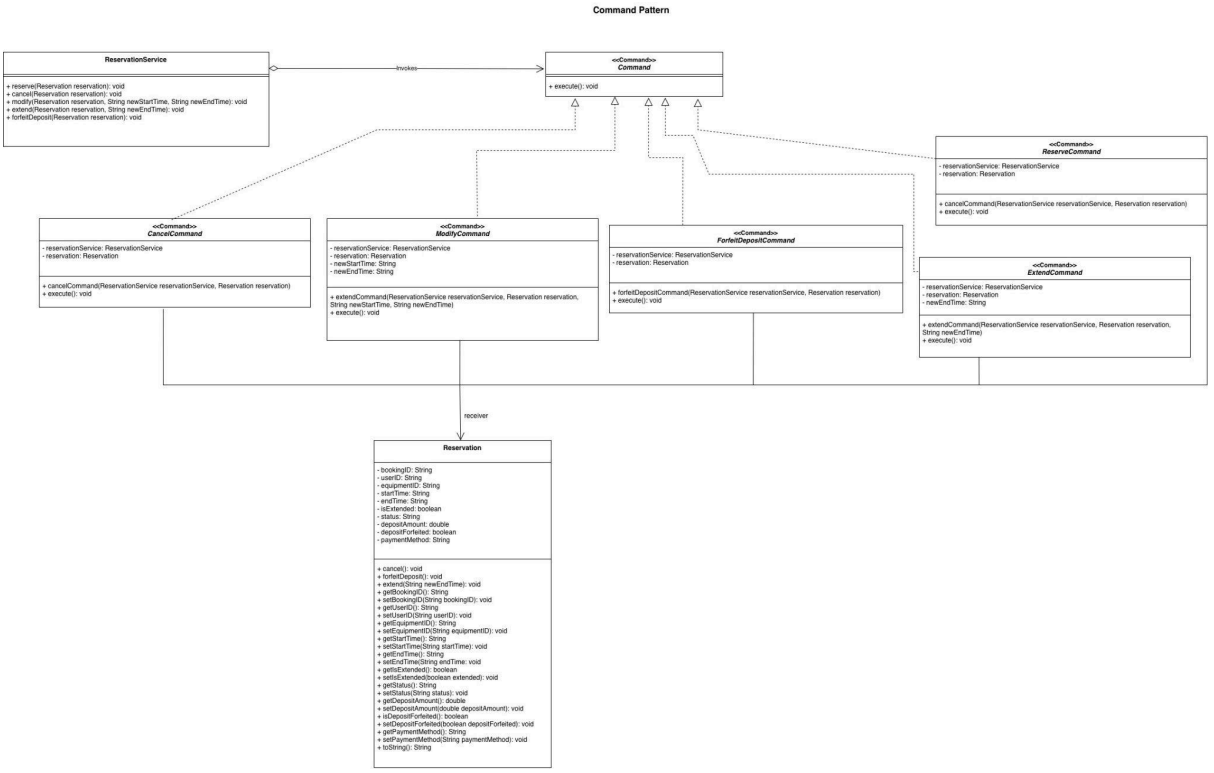
Task 1b: Class Diagrams

---

1. Booking System Singleton Pattern

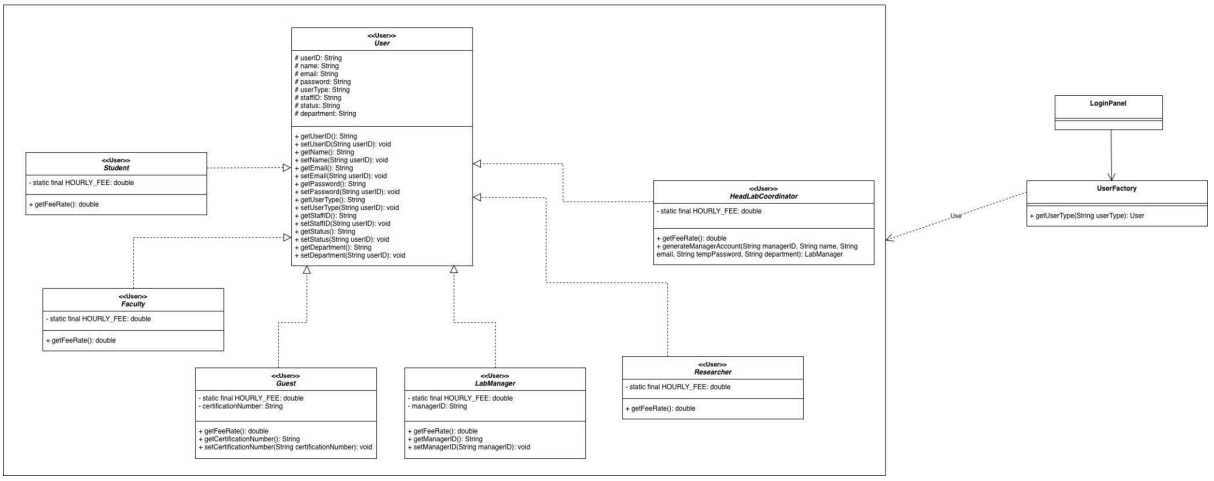


2. Command Pattern



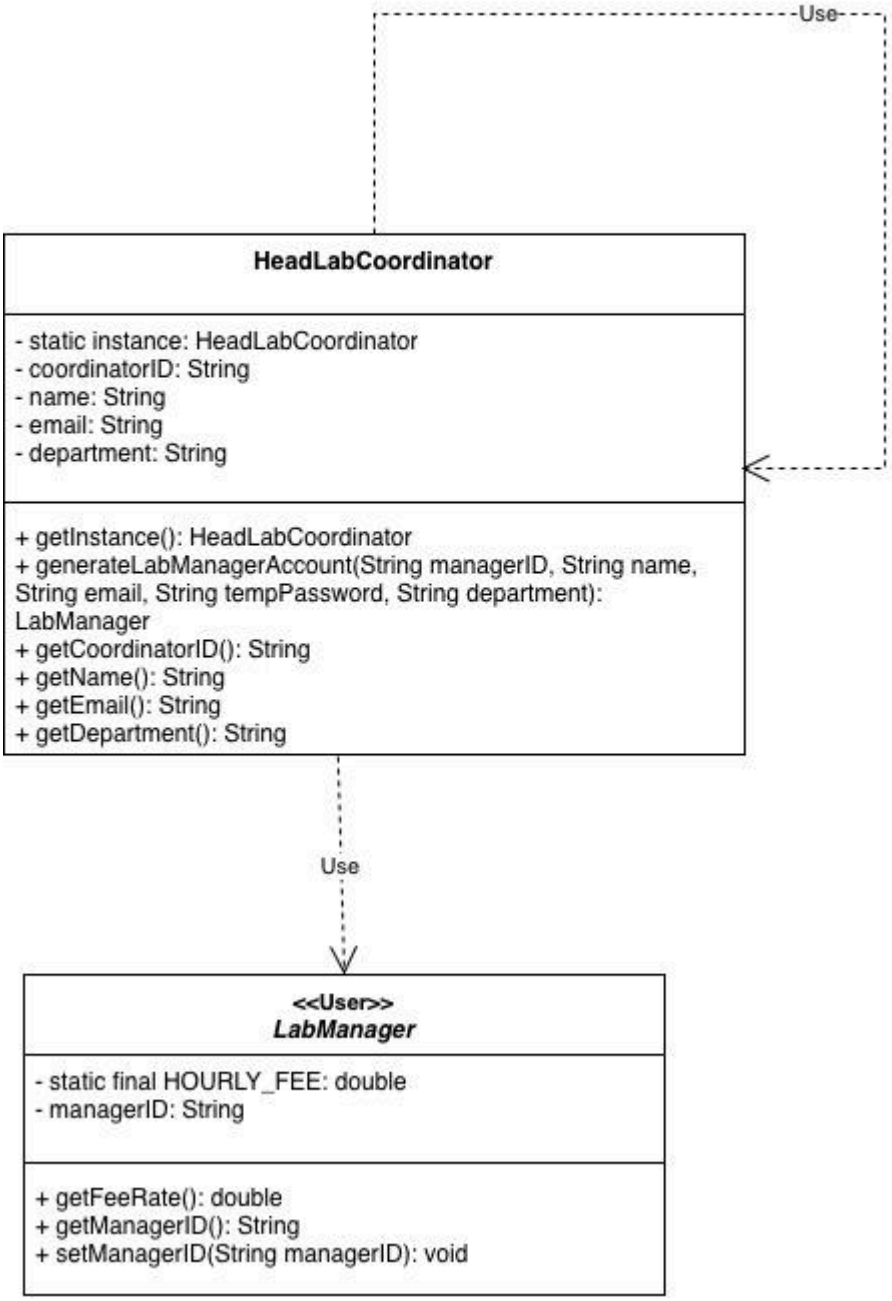
3. Factory Pattern

UserFactory Pattern

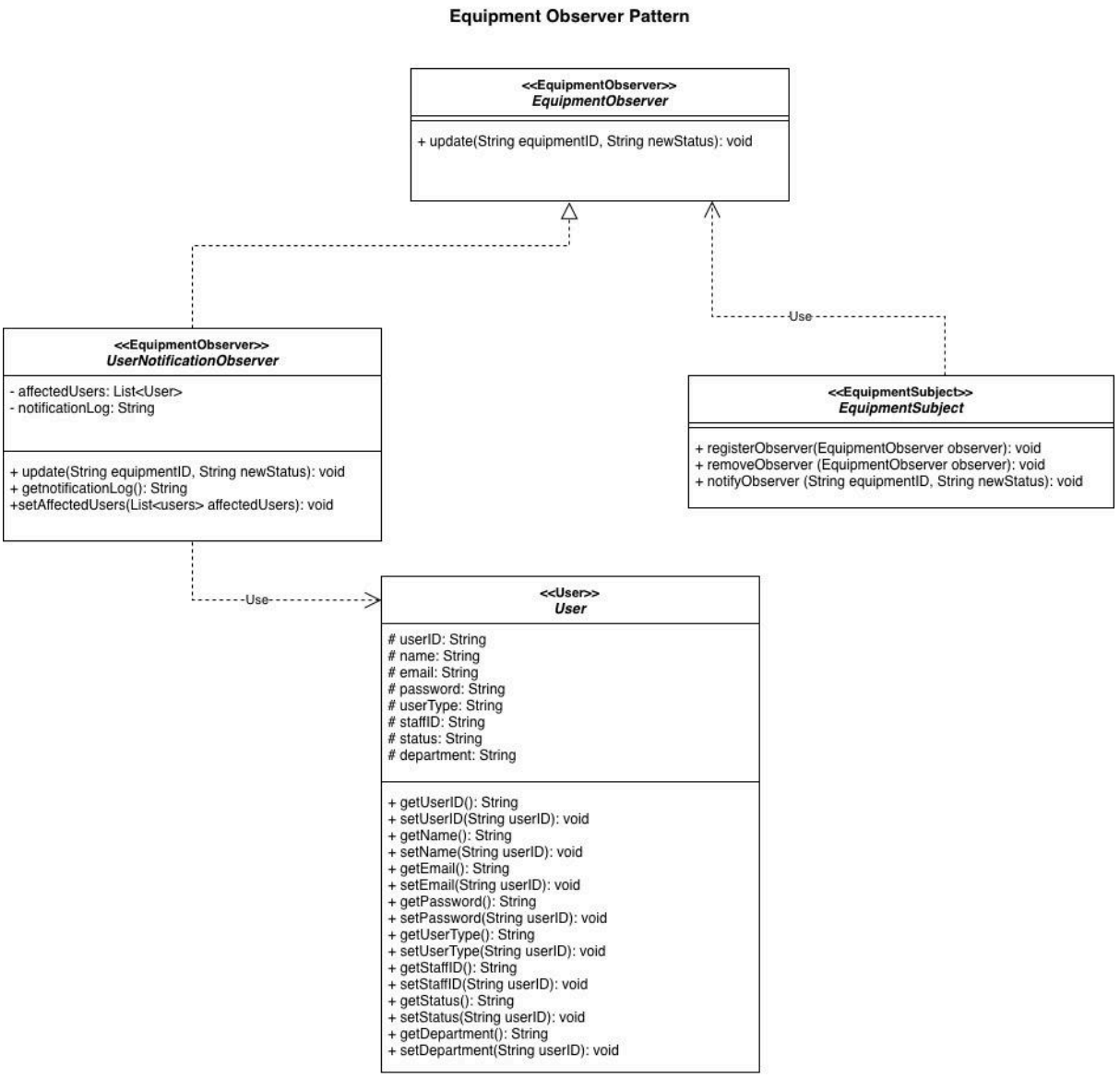


4. HeadLabCoordinator

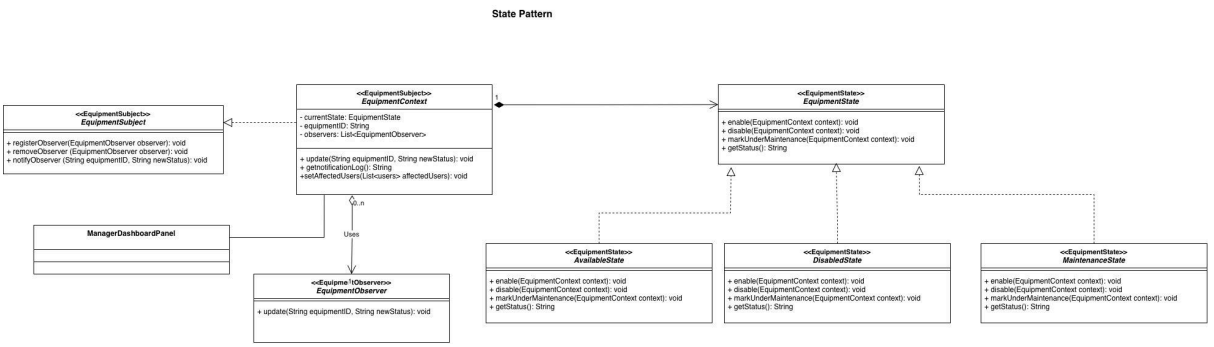
HeadLabCoordinator Singleton Pattern



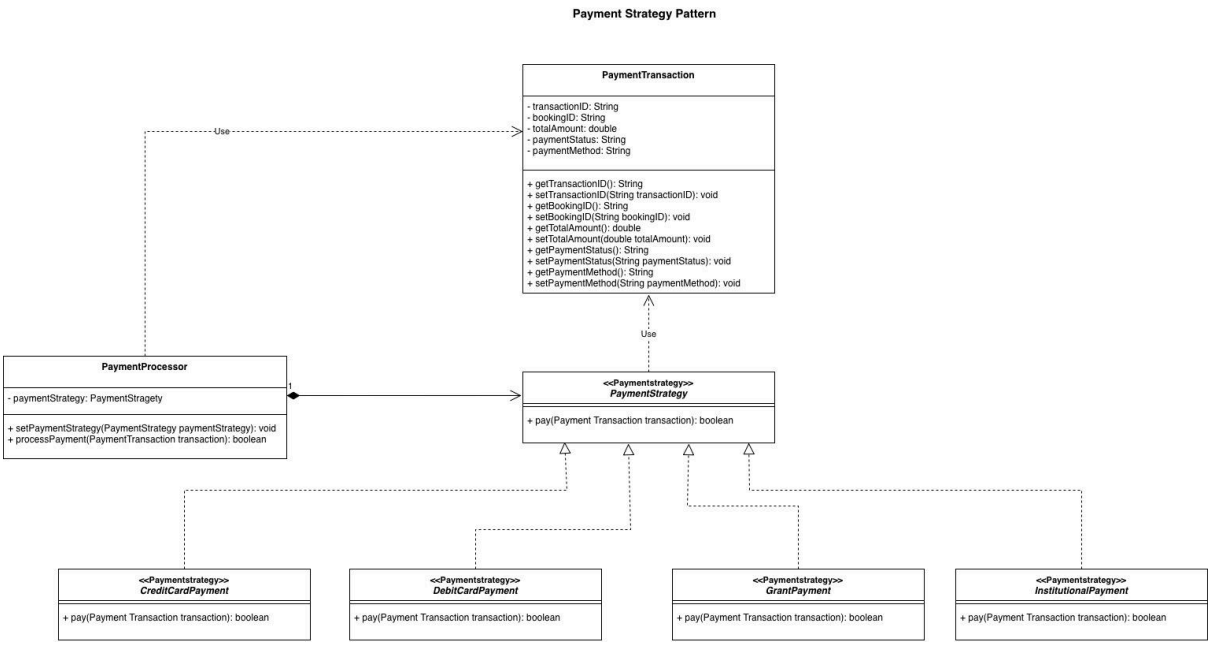
5. Observer Pattern



6. State Pattern



7. Strategy Pattern



Task 1c: How each requirement can be achieved

Req 1: Register + Login

Description: The system must allow users to register with specific roles (Student, Faculty, etc.) and authenticate existing users to grant access to their respective dashboards.

- **Logic & Workflow**: * The LoginPanel collects user details such as ID, name, email, and user type.
    - The Factory Pattern is used to instantiate the specific subclass of the abstract User class (e.g., Student, Faculty) based on the provided input.
    - During login, the system validates credentials against the CSV-simulated database.
    - Once authenticated, the system reconstructs the user object via the factory to redirect the user to the correct role-based dashboard.
- **Design Pattern Connection:** This is achieved through the Factory Method Pattern. By centralizing object creation in a UserFactory, the system ensures that adding new user types in the future does not require modifying the registration or login UI logic.
- **Key Implementation Evidence:** * UserFactory.createUser(type, ...) returns the appropriate subclass.
    - User abstract class defines common attributes for all roles.

Req 2: Head coordinator auto-generates manager account

Description: The system must ensure that only one Head Coordinator exists and that they alone have the authority to generate Lab Manager accounts.

- **Logic & Workflow:** * The system utilizes a HeadLabCoordinatorSingleton to restrict the coordinator role to a single instance.
    - The Coordinator invokes generateManagerAccount(), which internally uses the UserFactory to produce a LabManager object.
    - This ensures that manager accounts are created with the same consistency as standard user accounts while maintaining strict access control.
- **Design Pattern Connection:** This requirement uses both the Singleton Pattern and the Factory Pattern. The Singleton enforces a global point of access for the high-privileged Coordinator, while the Factory standardizes the creation of the LabManager role.
- **Key Implementation Evidence:** * HeadLabCoordinator.getInstance() ensures a unique coordinator instance.
    - UserFactory supports the "MANAGER" type for consistent reconstruction.

Req 3: Browse + Reserve Equipment

Description: Users must be able to view available equipment and confirm bookings, which are then stored consistently across the system.

- **Logic & Workflow:** * Users browse equipment data retrieved from the CSV files.
    - When a booking is confirmed, a Reservation object is initialized with a "CONFIRMED" status.
    - A ReserveCommand object is created to encapsulate the booking action.
    - The BookingSystem (a Singleton) stores the reservation, ensuring that all UI panels see the same real-time booking data.
- **Design Pattern Connection:** This requirement utilizes the Command Pattern and Singleton Pattern. The Command pattern decouples the GUI from the ReservationService logic, while the Singleton ensures a single, consistent source of truth for all bookings.
- **Key Implementation Evidence:** * ReserveCommand.execute() handles the service call.
    - BookingSystem.getInstance().addBooking(reservation) maintains the shared list.

Req 4: Deposit paid, forfeited if > 20 min late

Description: The system must track deposit payments and automatically forfeit deposits if a user is more than 20 minutes late.

- **Logic & Workflow:** * The Reservation model includes a depositAmount and a depositForfeited flag.
    - If a late arrival is detected, the system triggers a ForfeitDepositCommand.

    - The command calls ReservationService.forfeitDeposit(), which updates the internal state of the reservation to "FORFEITED" and sets the boolean flag.
- **Design Pattern Connection**: This requirement is primarily achieved through the Command Pattern. Encapsulating the forfeiture logic in a Command object allows the system to trigger this business rule from various parts of the app (e.g., a manager manual override or a timed check) consistently.
- **Key Implementation Evidence**: * ForfeitDepositCommand encapsulates the logic for penalizing late users.
    - reservation.forfeitDeposit() method updates the flags and status.

Req 5: Track equipment usage (sensor simulation)

Description: The system must monitor equipment status/activity through simulated sensor behaviour and notify relevant users when equipment status changes affect their bookings.

- **Logic & Workflow:**
    - EquipmentContext acts as the central object representing a piece of equipment and maintains a list of observers.
    - When equipment status changes, the context calls notifyObservers(equipmentID, newStatus).
    - The Observer Pattern is used so that notification behaviour is separated from the equipment logic itself.
    - UserNotificationObserver receives these updates and simulates notifying users whose reservations may be affected by the new equipment status.
    - This allows the system to track usage/status changes in real time without directly coupling equipment management to user notification logic.

- **Design Pattern Connection:** This is achieved through the Observer Pattern. By allowing EquipmentContext to act as a subject and UserNotificationObserver to act as an observer, the system can respond automatically to equipment activity or status changes without hard-coding notification behaviour into the equipment class.

- **Key Implementation Evidence:**
    - EquipmentContext stores and manages observers through the EquipmentSubject interface.
    - notifyObservers(...) is called whenever equipment status changes. UserNotificationObserver.update(...) handles the simulated notification logic for affected users.

Req 6: Add/enable/disable equipment

Description: The system must allow managers to add equipment and control its status (AVAILABLE, DISABLED, MAINTENANCE).

- **Logic & Workflow:**
  - Each equipment is represented by EquipmentContext, which stores its current state.
  - The State Pattern is used to manage valid transitions between states.
  - Actions (enable(), disable(), markUnderMaintenance()) are delegated to the current state.
  - Each state defines its own valid transitions (e.g., Available → Disabled).
  - setState(...) also triggers notifyObservers().
- **Design Pattern Connection:** This is achieved through the State Pattern. It removes conditional logic by delegating behaviour to state classes and improves maintainability.
- **Key Implementation Evidence:**
  - EquipmentState defines state methods.
    Concrete states implement transitions.
  - EquipmentContext delegates calls to the current state.

Req 7: Assign equipment ID/description/location

Description: The system must store identifying information for each equipment item, including its ID, description, and location.

- **Logic & Workflow:**
  - Each equipment item is represented by an EquipmentContext object.
  - When equipment is added, the manager provides its ID, description, and location.
  - These values are stored as attributes of the equipment object and used whenever the system displays or manages that equipment.
  - This ensures that each equipment item can be uniquely identified and tracked in the system.
- **Design Pattern Connection:** This requirement is most closely associated with the State Pattern, since EquipmentContext is the central class representing each equipment item. However, Req7 is mainly achieved through object attributes and encapsulation rather than state transitions.
- **Key Implementation Evidence:**
  - EquipmentContext represents each equipment item.
  - Equipment details such as ID, description, and location are stored as part of the equipment object.

    - These attributes are used by the system when displaying, updating, or reserving equipment.

Req 8: Provide ID, modify/cancel booking

Description: The system must allow users to provide a booking ID and then modify or cancel an existing reservation.

- **Logic & Workflow:**
    - Each reservation stores a unique bookingID used to identify the booking.
    - When a user enters the booking ID, the system locates the corresponding Reservation.
    - The Command Pattern is used to perform booking actions without the GUI directly changing the model.
    - ModifyCommand updates the reservation start and end times through ReservationService.modify(...).
    - CancelCommand updates the reservation status to "CANCELLED" through ReservationService.cancel(...).
- **Design Pattern Connection:** This is achieved through the Command Pattern. By encapsulating modify and cancel actions as command objects, the system keeps the GUI separate from reservation business logic.
- **Key Implementation Evidence:**
    - Reservation stores bookingID and booking details.
    - ModifyCommand.execute() calls reservationService.modify(...).
    - CancelCommand.execute() calls reservationService.cancel(...).

Req 9: Extend reservation if equipment free

Description: The system must allow users to extend an existing reservation only if the equipment is available for the requested extended time.

- **Logic & Workflow:**
  The reservation is first identified using its booking ID.
    - The system checks shared booking records in BookingSystem to ensure no conflicting reservation exists for that equipment.
    - If the equipment is free, the Command Pattern is used to execute the extension.
    - ExtendCommand calls ReservationService.extend(...), which updates the reservation end time and marks the reservation as extended.
    - Because all bookings are stored in the singleton BookingSystem, availability checks use one shared reservation list.
- **Design Pattern Connection:** This is achieved through both the Singleton and Command patterns. Singleton ensures all availability checks use the same booking data, while Command encapsulates the extension action.

- **Key Implementation Evidence:**
    - BookingSystem.getInstance() provides shared access to reservations.
    - ExtendCommand.execute() calls reservationService.extend(...).
    - Reservation.extend(...) updates endTime and sets isExtended = true.

Req 10: Multiple Payment Methods

Description: The system must support multiple payment methods, including credit card, debit, institutional account, and research grant.

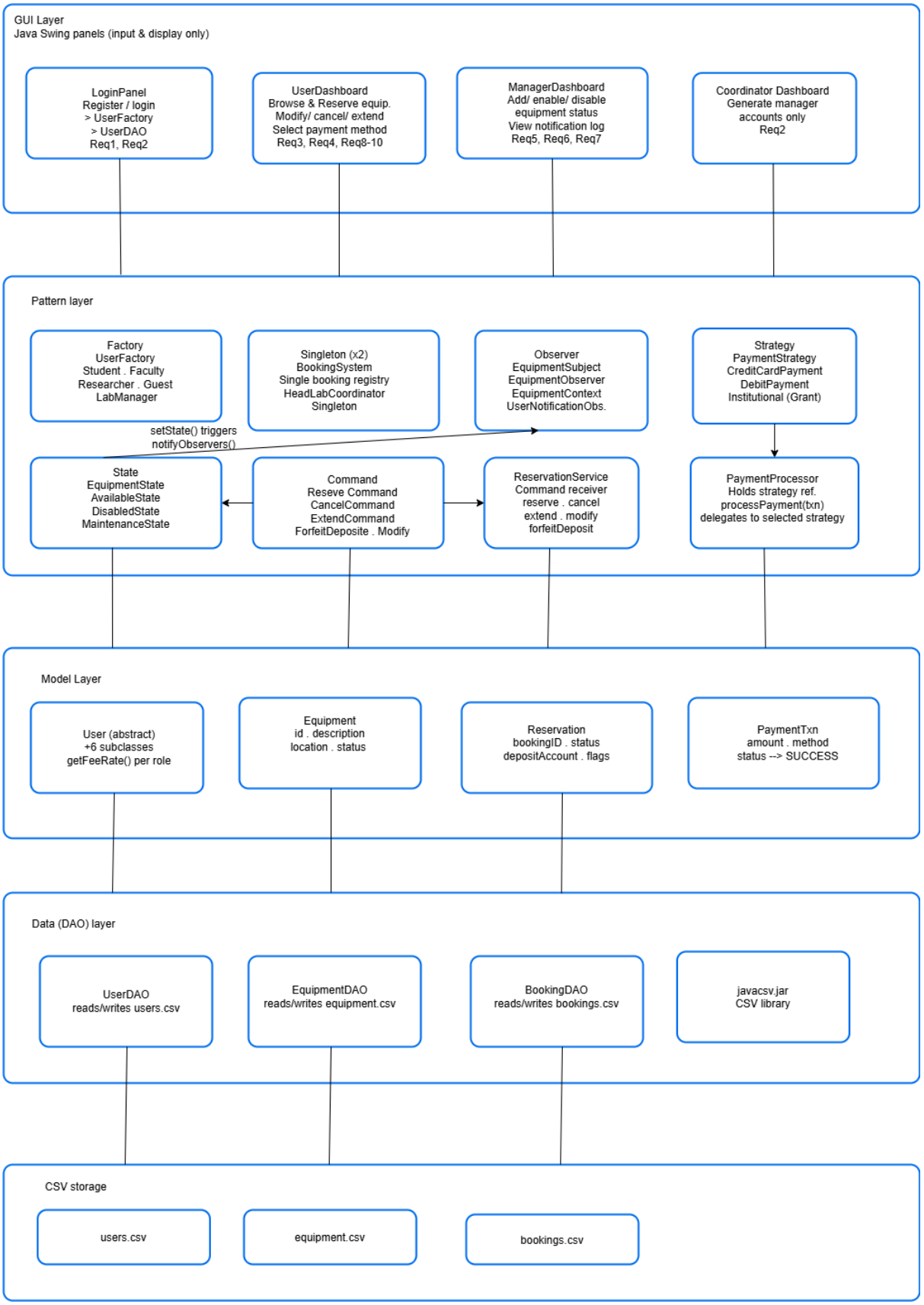
- **Logic & Workflow:**
    - The user selects a payment method from the UI (e.g., dropdown).
    - The Strategy Pattern is used to handle different payment behaviours.
      The system selects the appropriate PaymentStrategy implementation (e.g., CreditCardPayment, DebitPayment).
    - PaymentProcessor is initialized with the selected strategy and calls processPayment(transaction).
    - The selected strategy processes the payment and updates the transaction status to "SUCCESS".
- **Design Pattern Connection:** This is achieved through the Strategy Pattern. By encapsulating each payment method in its own class, the system avoids conditional logic and allows new payment methods to be added easily.
- **Key Implementation Evidence:**
    - PaymentStrategy defines pay(transaction).
    - PaymentProcessor.processPayment(...) delegates to the selected strategy.
    - Concrete strategies (CreditCardPayment, DebitPayment, etc.) implement payment logic and update status.

Task 1d : Component Diagram

The system is decomposed into five main components:

Component	Responsibilities	Depends On
GUI Layer	LoginPanel, UserDashboardPanel, ManagerDashboardPanel, CoordinatorDashboardPanel: present information and handle user input	Service/Pattern Layer, Model Layer

Model Layer	User hierarchy, Equipment, Reservation, Deposit, PaymentTransaction: plain data objects with no dependencies on other layers	None
Data Layer	UserDAO, EquipmentDAO, BookingDAO: reads and writes CSV files using javacsv.jar	Model Layer
Pattern Layer	All pattern classes (factory/, singleton/, observer/, state/, strategy/, command/) :implements business logic using design patterns	Model Layer, Data Layer
CSV Storage	users.csv, equipment.csv, bookings.csv: flat-file persistent storage simulating a database	None


GUI Layer
Java Swing panels (input & display only)
LoginPanel
Register / login
> UserFactory
> UserDAO
Req1, Req2
UserDashboard
Browse & Reserve equip.
Modify/ cancel/ extend
Select payment method
Req3, Req4, Req8-10
ManagerDashboard
Add/ enable/ disable
equipment status
View notification log
Req5, Req6, Req7
Coordinator Dashboard
Generate manager
accounts only
Req2
Pattern layer
Factory
UserFactory
Student . Faculty
Researcher . Guest
LabManager
Singleton (x2)
BookingSystem
Single booking registry
HeadLabCoordinator
Singleton
Observer
EquipmentSubject
EquipmentObserver
EquipmentContext
UserNotificationObs.
Strategy
PaymentStrategy
CreditCardPayment
DebitPayment
Institutional (Grant)
setState() triggers
notifyObservers()
State
EquipmentState
AvailableState
DisabledState
MaintenanceState
Command
Reseve Command
CancelCommand
ExtendCommand
ForfeitDeposite . Modify
ReservationService
Command receiver
reserve . cancel
extend . modify
forfeitDeposit
PaymentProcessor
Holds strategy ref.
processPayment(txn)
delegates to selected strategy
Model Layer
User (abstract)
+6 subclasses
getFeeRate() per role
Equipment
id . description
location . status
Reservation
bookingID . status
depositAccount . flags
PaymentTxn
amount . method
status --> SUCCESS
Data (DAO) layer
UserDAO
reads/writes users.csv
EquipmentDAO
reads/writes equipment.csv
BookingDAO
reads/writes bookings.csv
javacsv.jar
CSV library
CSV storage
users.csv
equipment.csv
bookings.csv

Task 1e: Justification for Component Decomposition

The system is divided into different components to improve maintainability, extensibility, and separation of concerns**.** By separating the system into smaller parts, each component can focus on a specific responsibility, making the system easier to understand and modify.

Component Breakdown

Presentation Component (UI):
This includes LoginPanel and the different dashboards. It is only responsible for handling user input and displaying information. It does not contain business logic, so changes to the UI do not affect the rest of the system.

User Management Component:
This component handles registration, login, and user roles using the Factory Pattern (UserFactory). It is separated so that user-related logic is independent from booking or equipment functionality.

Booking Component:
This includes BookingSystem (Singleton), ReservationService, and command classes. It manages all booking operations such as reserving, cancelling, and extending. Keeping this logic together ensures that booking rules are applied consistently.

Equipment Component:
This component manages equipment details and status using the State Pattern (EquipmentContext). It is separated because equipment behaviour is different from user or booking logic.

Notification Component:
This uses the Observer Pattern to notify users when equipment status changes. Separating this prevents the equipment logic from being tightly coupled with notification handling.

Payment Component:
This component handles payments using the Strategy Pattern. It allows different payment methods to be added easily without changing the booking system.

Component Interaction & Logic Flow

- **UI to Service:** The UI does not directly modify data. Instead, it calls service or command classes to perform actions.
- **Booking & Equipment:** The Booking component checks equipment availability through the Equipment component before confirming reservations.

- **Notifications:** When equipment status changes (e.g., set to maintenance), the Equipment component notifies the Notification component, which then alerts affected users.
- **Payments:** The Booking component interacts with the Payment component to process transactions.

Architectural Justification

- **Loose Coupling:** Components communicate through clear interfaces and design patterns, so changes in one component have minimal impact on others.
- **High Cohesion:** Each component has a clear responsibility (e.g., booking, payment, equipment), making the system easier to debug and test.
- **Scalability:** The design allows future improvements, such as replacing CSV storage with a database, without affecting other parts of the system.

Task 4: AI-Assistant Adaptation Report

This report describes how AI assistants were used throughout Deliverable 2: the redesign and Java implementation of the YorkU Lab Equipment Reservation Platform. Our group used ChatGPT (GPT-4o) for initial architectural brainstorming and to select the most suitable design pattern. Also, we used Claude (Sonnet 4.6) to refine our implementation logic (GUI: Java Swing components) and verify the correctness of our UML model.

Throughout the project, we treated AI output as a starting point, evaluating every suggestion against the specified project requirements (Req 1 - Req 10), the course lecture notes on design patterns, and our own understanding of the topics. Where the AI was helpful or wrong, we include the exact prompts and explain what was accepted, or we document the disagreement and how it was resolved.

Role of AI in tasks 1 and 2:

This section covers AI usage across four areas: design pattern selection, class diagram redesign, component decomposition, and implementation.

Design Pattern Selection

AI tool: ChatGPT (GPT-4o)

Prompt:
"A Java Lab Equipment Reservation System has user types with different hourly rates, equipment with statuses that update (available, disabled, maintenance), bookings that can be created/cancelled/extended/modified, a deposit that forfeits if the user is more than 20 minutes late, and payment via debit, credit, institutional account, or research grant. Suggest at least 6 or more design patterns and justify each one?"

AI Output:
The AI suggested eight types of design patterns with a brief justification for each tied to the requirements. The output is as follows:

- Singleton for the booking management system and head lab coordinator
- Factory for user creation by type
- Strategy for payment methods
- Observer for equipment status change notifications
- Decorator for booking extension and additional charges
- Command for booking operations
- State for equipment status transitions.
- Template Method to validate users, check equipment availability, etc.

Our Evaluation:
Based on the eight patterns suggested, we chose only the six patterns that matched our planned direction and mapped well according to the requirements. Therefore, we accepted the list as the baseline, but our understanding of how each pattern interacts with others required significant development beyond the AI's description. The six patterns we decided on are as follows: Singleton, Factory, Strategy, Observer, Command, and State.

One important decision the AI did not address was whether to combine State and Observer into a single class. Our final implementation has the EquipmentContext implement both EquipmentSubject (Observer pattern) and host the State pattern context. The AI treated these as two independent patterns, but we recognized that equipment status changes and the notifications triggered by those changes are tightly coupled; separating them into different classes would require one to call the other, creating unnecessary dependency. So by having EquipmentContext manage both, a state transition (triggered by enable(), disable(), or markUnderMaintenance()) automatically notifies observers through the same setState() method. This was a deliberate cohesion decision, not suggested by the AI.

Class Diagram Redesign

AI tool: Claude (Sonnet 4.6)

Prompt:
After uploading our class diagram from D1, we prompt Claude to "update the class diagram to incorporate our six design patterns (Singleton(BookingSystem, HeadLabCoordinator), Strategy(PaymentStrategy), Factory(UserFactory), Command(reserve, cancel, extend, modify, and forfeitDeposit), Observer(EquipmentSubject/EquipmentObserver), State(EquipmentContext with AvailableState, DisabledState, MaintenanceState). The User hierarchy includes Student, Faculty, Researcher, Guest, LabManager, and HeadLabCoordinator, which extend the abstract User base. Show all the relationships and interfaces?"

AI Output:
Claude produced a detailed class diagram description listing all classes, their attributes and methods, and the relationships between them. It identified the <<interface>> type for Command, PaymentStrategy, EquipmentObserver, EquipmentSubject, and EquipmentState. It showed inheritance arrows from six user subtypes to User, and dependency arrows from the Command concrete classes to ReservationServices.

Our Evaluation:
We noticed that Claude's diagram placed the Observer registration logic in a separate NotificationService class rather than inside EquipmentContext. We rejected this because it added a class whose sole purpose was to hold an observer list that belonged to

EquipmentContext (the class that already owns the state being observed). We merged the Observer subject responsibilities into EquipmentContext directly (it implements EquipmentSubject) and verified this is a standard Observer pattern variant where the subject is also the data source.

Component Decomposition

AI tool: ChatGPT (GPT-4o)

Prompt:
"Suggest a component decomposition for a Java Swing application that implements the YorkU Lab Equipment Reservation System. The system has user authentication, equipment management, booking management, payment processing, and status notifications. How would these be broken down into components? And what are the dependencies between the components, and what interfaces should each component display?"

AI Output:
ChatGPT suggested 5 components: Presentation Layer (GUI panels), Business Logic Layer (Pattern classes), Data Access Layer (DAO classes), Model Layer (Plain Old Java classes), and a Notification Component (Observer classes). It described dependencies flowing from presentation to business logic to data access to model, with notification sitting alongside business logic and receiving events from it.

Our Evaluation:
We accepted this as a structural framework and adopted the layered decomposition, mapping it directly to our package structure: gui/, pattern/, data/, and model/. However, the AI described the notification component as independent, which we corrected in our implementation. The notification logic lives inside the Observer pattern classes (pattern/observer/), which EquipmentContext calls in the pattern/ layer. There is no separate notification component as it is integrated into the Business Logic Layer.

We also justified the decomposition in terms of the system requirements: the Data Access Layer (BookingDAO, EquipmentDAO, UserDAO) isolates CSV persistence so the rest of the system does not need to know how data is stored, making it straightforward to replace CSV files with a database in the future. The pattern/ package groups all design pattern implementations together, making it easy to locate and demonstrate each pattern during the demo.

Implementation

AI tool: Claude (Sonnet 4.6)

Prompt #1: For the Command Pattern (Booking Operations)

“We have a ReservationService class that handles reserve, cancel, extend, modify, and forfeitDeposit operations on the Reservation objects. Implement the Command pattern so that each operation becomes a concrete command class. And show how they’re used in the GUI?“

AI Output:

Claude produces a Command interface with a single execute() method, five concrete command classes (ReserveCommand, CancelCommand, ModifyCommand, ExtendCommand, ForfeitDepositCommand), and a BookingInvoker class that holds a queue of commands to be executed in sequence. The GUI usage example, called invoker.addCommand(new ReserveCommand(svc, r)) followed by invoker.executeAll() was suggested to be implemented.

Our Evaluation:

We implemented all five concrete command classes as suggested, but did not implement the BookingInvoker class. Instead, we had the GUI call commands directly, for example, in UserDashboardPanel, new ReserveCommand(svc, r).execute() is called. This means the Command pattern technically lacks a formal Invoker, but this was a conscious tradeoff we decided on to avoid unnecessary complexity in the GUI layer.

Prompt #2: For Swing GUI Java application (ManagerDashboardPanel)

"How should we use the State pattern via EquipmentContext to change the equipment status in the Manager Dashboard equipment management tab when the manager clicks 'Update Status'?"

AI Output:

Claude explained that EquipmentContext manages the in-memory state transition while EquipmentDAO manages persistence, and that they are separated intentionally. It showed a call sequence in this order: (1) create an EquipmentContext from the current equipment status; (2) call enable(), disable(), or markUnderMaintenance() to transition the state; (3) call getStatusString() to get the new equipmentDAO.updateEquipment(e) to update the list; (6) call equipmentDAO.save() to write back to CSV.

Our Evaluation:

This guidance clarified a design tension in our system as it describes how EquipmentContext is a runtime state machine and is not persisted directly, while Equipment is the data object

that maps to the CSV row. The AI identified that these serve different purposes and should remain separate. Our final ManagerDashboardPanel follows this sequence exactly.

Arguments and Resolution:

Disagreement: Observer should be a separate service class

AI suggestion - To place observer registration and notification logic in a separate NotificationService class. Then have EquipmentContext call notificationService.notify(equipmentID, newStatus) after each state transition.

Technical Issue Identified - This creates an unnecessary circular dependency, where EquipmentContext calls NotificationService, but NotificationService would need to know which observers are subscribed to which equipment. The Observer pattern defines the subject as the entity that holds the observer list, not another class or a third-party class.

Reasoning Process - A separate NotificationService adds a class whose only job is to hold a list that belongs naturally to EquipmentContext. For a system of this size, the added indirection reduces clarity without any real benefit or flexibility.

Resolution - We rejected this suggestion because EquipmentContext implements EquipmentSubject and manages its own observer list. The combined State + Observer in EquipmentContext is a deliberate cohesion decision, since a state transition always triggers a notification; hence, keeping both responsibilities in the same class was the cleaner design.

Verification of AI Responses:

We apply these four verification checks to each significant AI suggestion before accepting it:

1. Requirement accuracy: We checked whether the suggestion addressed the relevant specified requirement (Req 1 - Req 10). And if it did not, we mapped it ourselves or rejected it.
2. UML consistency: We verified that the AI's class diagram descriptions used correct UML notation, interfaces marked with <<interface>>, inheritance shown correctly, dependencies and associations represented accurately.
3. Pattern correctness: We cross-referenced each suggestion against the design pattern definitions covered in the course lecture slides, checking that all required participants were present, relationships were correct, and patterns were not misnamed.

4. Integration: Before accepting implementation suggestions, at least one group member integrated the code into the project and confirmed it compiled, produced the expected output, and did not break existing functionality.

Concrete Example:

Claude suggested implementing a BookingInvoker class as part of the Command pattern, a class that holds a reference to a command and triggers execution via an execute() method.

We verified this against the Command pattern as taught in the course. The lecture slides identify four participants: Client, Invoker, Command (interface), and Receiver. The Invoker holds a reference to a command object and calls execute() on it without knowing what the command does. The Receiver (in our case, ReservationService) performs the actual action. The lecture's code example confirms this, as the Invoker has a setCommand() method and an execute() method that delegates to the command.

We also verified our Observer pattern implementation against the course notes, which identify four key players: Subject, Observer, ConcreteSubject, and ConcreteObserver. The Subject maintains a list of observers and provides attach and detach interfaces. The Observer defines the update interface. The ConcreteSubject stores the state of interest and notifies observers when it changes. The ConcreteObserver implements the update interface to react to those changes. In our system, EquipmentContext acts as the ConcreteSubject, EquipmentSubject is the Subject interface, EquipmentObserver is the Observer interface, and UserNotificationObserver is the ConcreteObserver. We verified that all four participants are present and correctly structured, confirming the pattern is fully implemented as defined in the course.

For the Command pattern, our implementation has the Command interface, five concrete commands, and ReservationService as the Receiver, all present in the lecture's pattern structure. However, our GUI panels call commands directly (for example, new ReserveCommand(svc, r).execute() in UserDashboardPanel), meaning the panel acts as an informal invoker rather than delegating to a dedicated Invoker class. Claude's suggestion to include a formal BookingInvoker was therefore verified as correct with respect to the course notes definition, but was partially rejected as we prefer to have an improved code clarity while keeping it flexible as well. A complete implementation would introduce a BookingInvoker that the dashboard panels delegate to, matching the Client -> Invoker -> ConcreteCommand -> Receiver structure shown in the lecture notes.